

The system relies on a two-agent architecture composed of an Analyzer Agent and a Verifying Agent, both coordinated through a fixed, linear pipeline that ensures consistency, modularity, and clear separation of responsibilities. Each agent operates with its own defined role, input format, and output format, which allows the system to maintain a high degree of organization and traceability as data moves from one stage to the next. This structure not only improves reliability but also makes the system easier to maintain, extend, and debug, since each agent's behavior is isolated and predictable.

The Analyzer Agent serves as the system's primary intelligence-gathering and interpretation component. It is responsible for taking the user's initial inputs, which include a written product description and, optionally, an uploaded product image, and transforming them into a structured understanding of the item the user is trying to match. To accomplish this, the Analyzer Agent extracts key attributes such as the product's category, brand, intended use, formulation or material characteristics, and any distinctive features that might influence similarity. Once the product has been conceptually modeled, the Analyzer Agent conducts a search-grounded analysis to gather real-time information about comparable products available online. This includes scanning for items that share functional characteristics, ingredient profiles, aesthetic qualities, or performance claims. The agent then compiles its findings into a structured dataset that lists potential dupes and alternatives, complete with product names, prices, retailer sources, feature comparisons, and ingredient or material similarities. This dataset is formatted as JSON to ensure that it can be passed cleanly and unambiguously to the next stage of the pipeline.

The Verifying Agent functions as the system's quality-control and analytical refinement layer. It receives the JSON output from the Analyzer Agent and evaluates the proposed dupes for accuracy, relevance, and completeness. This evaluation includes checking whether the suggested alternatives genuinely align with the user's original description and image, verifying ingredient or feature similarities, and ensuring that the matches are not superficial or misleading. The Verifying Agent applies quantitative scoring methods, computing similarity scores on a 0–1 scale based on the degree of overlap between the original product and each candidate. It also calculates value scores by dividing similarity by price, highlighting options that offer strong equivalence at a lower cost. During this process, the Verifying Agent filters out low-quality, irrelevant, or hallucinated results, ensuring that only credible and meaningful recommendations remain. The final output is a polished block of HTML content that includes headings, paragraphs, lists, and explanatory text, making it suitable for direct rendering on the webpage. Once the Verifying Agent completes its analysis, it signals task completion by calling `system_objective_fulfilled`.

The system's coordination mechanism follows a fixed pipeline structure consisting of four stages: User Input, Analyzer Agent, Verifying Agent, and Final HTML Output. This linear flow ensures that each agent receives the correct inputs in the correct order and that the output of one agent becomes the input of the next without ambiguity. A controller oversees the entire

sequence, passing data between agents, monitoring for errors, and ensuring that the workflow progresses smoothly. Communication between agents is strictly structured: the Analyzer Agent outputs JSON, which the Verifying Agent consumes, and the Verifying Agent outputs HTML, which the webpage renders directly. This separation of concerns ensures that each agent focuses exclusively on its designated responsibilities without overlapping functionality or interfering with other components of the system.

To maintain reliability, the system incorporates safeguards that prevent low-quality or incomplete outputs from reaching the user. If the Analyzer Agent returns insufficient data, such as missing product attributes, unclear matches, or empty search results, the Verifying Agent flags the issue. The controller then triggers a re-run of the Analyzer Agent with refined instructions designed to improve the quality of the results. This loop may occur up to two times to avoid infinite retries while still allowing the system to recover from weak initial outputs. If, after these retries, the Verifying Agent determines that the Analyzer's output remains fundamentally unreliable, the system produces a fallback message explaining that no suitable duplicates or alternatives could be identified. This approach ensures transparency, prevents misleading recommendations, and maintains user trust by avoiding the presentation of uncertain or fabricated results.